

Масштабируемые методы решения задачи Kernel Ridge Regression

Ядерный холецкий
Матросова Неонила
Максим Зориков

Постановка задачи Kernel Ridge Regression

Пусть $\mathcal{H}_K$ — воспроизводящее гильбертово пространство (RKHS), порожденное ядром $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$.

Задача Kernel Ridge Regression:

$$f^* = \arg \min_{f \in \mathcal{H}_K} \left\{ \mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2 + \lambda \|f\|_{\mathcal{H}_K}^2 \right\}$$

Свойства решения:

1. По теореме Репрезентатора: $f^*(\cdot) = \sum_{i=1}^n \alpha_i^* K(\cdot, \mathbf{x}_i)$
2. Оптимальные коэффициенты: $\boldsymbol{\alpha}^* = (\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}$
3. Матричная форма: $(\mathbf{K} + \lambda n \mathbf{I}) \boldsymbol{\alpha} = \mathbf{y}$

В чем проблема?

Прямое решение находится за $O(n^3 + n^2 d)$ и требует $O(n^2)$ памяти. Это достаточно дорогое решение, которое хотелось бы сделать дешевле.

Ключевые подходы:

Restricted KRR

Preconditioned Conjugate Gradient

Restricted KRR

В памяти потребуется

хранить всего $O(m * n + m^2)$

А решение займет $O(m^3)$

Качество из-за

неиспользования

полной выборки.

Restricted KRR: ищем решение в подпространстве порожденном $m \ll n$ опорными точками $\{\mathbf{z}_j\}_{j=1}^m$

$$f(\mathbf{x}) = \sum_{j=1}^m \beta_j K(\mathbf{x}, \mathbf{z}_j), \quad \boldsymbol{\beta} \in \mathbb{R}^m$$

$$\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta} \in \mathbb{R}^m} \left\| \mathbf{y} - \mathbf{K}_{nm} \boldsymbol{\beta} \right\|_2^2 + \lambda \boldsymbol{\beta}^\top \mathbf{K}_{mm} \boldsymbol{\beta}$$

Где:

- $\mathbf{K}_{nm} \in \mathbb{R}^{n \times m}$: $[\mathbf{K}_{nm}]_{ij} = K(\mathbf{x}_i, \mathbf{z}_j)$
- $\mathbf{K}_{mm} \in \mathbb{R}^{m \times m}$: $[\mathbf{K}_{mm}]_{ij} = K(\mathbf{z}_i, \mathbf{z}_j)$
- $\boldsymbol{\beta}^* = (\mathbf{K}_{nm}^\top \mathbf{K}_{nm} + \lambda \mathbf{K}_{mm})^{-1} \mathbf{K}_{nm}^\top \mathbf{y}$

Preconditioned Conjugate Gradient

$$\text{Решаем: } \mathbf{A}\boldsymbol{\alpha} = \mathbf{b}, \quad \mathbf{A} = \mathbf{K} + \lambda n \mathbf{I}$$

$$\text{Предобусловленная система: } \mathbf{P}^{-1} \mathbf{A} \boldsymbol{\alpha} = \mathbf{P}^{-1} \mathbf{b}$$

Где $\mathbf{P}$ — предобусловитель, такой что:

1. $\mathbf{P} \approx \mathbf{A}$
2. $\mathbf{P}^{-1}$ легко вычисляется
3. $\kappa(\mathbf{P}^{-1} \mathbf{A}) \ll \kappa(\mathbf{A})$ (число обусловленности улучшается)

$$\mathbf{P}^{-1} = (\widehat{\mathbf{K}} + n\lambda \mathbf{I})^{-1}$$

где $\widehat{\mathbf{K}} \in \mathbb{R}^{n \times n}$ — низкоранговая аппроксимация матрицы $\mathbf{K}$

Итеративные методы обычно сходятся быстрее аналитических решений, правильное предобуславливание этому способствует.

Как выбрать m опорных векторов?

- RLS (Ridge Leverage)

Ridge Leverage Score (RLS) точки i :

$$\ell_i^\lambda = [\mathbf{K}(\mathbf{K} + \lambda \mathbf{I})^{-1}]_{ii}$$

где:

- $\mathbf{K} \in \mathbb{R}^{n \times n}$ — матрица ядра
- $\lambda > 0$ — параметр регуляризации
- $(\mathbf{K} + \lambda \mathbf{I})^{-1}$ — регуляризованная обратная

$$p_i = \frac{\ell_i^\lambda}{\sum_{j=1}^n \ell_j^\lambda}$$

- RP Cholesky
- K-means

В качестве опорных векторов берем центры m кластеров.

RP Cholesky

Randomized Pivoted Cholesky

$\mathbf{d} \leftarrow \text{diag}(\mathbf{K})$ ▶ остаточная диагональ

$\mathbf{L} \leftarrow \mathbf{0}_{n \times m}$ ▶ инициализация

for $j = 1$ to m :

Выбрать i с $\mathbb{P}(i) = \frac{d_i}{\|\mathbf{d}\|_1}$

$\ell_{i,j} \leftarrow \sqrt{d_i}$

$\mathbf{L}_{:,j} \leftarrow \frac{1}{\ell_{i,j}} (\mathbf{K}_{:,i} - \mathbf{L}_{:,j-1} \mathbf{L}_{i,j-1}^\top)$

$\mathbf{d} \leftarrow \mathbf{d} - \mathbf{L}_{:,j} \odot \mathbf{L}_{:,j}$

$$\mathbf{K} \approx \mathbf{L}\mathbf{L}^\top, \quad \text{rank}(\mathbf{L}) = m$$

Кроме выбора опорных векторов еще и вычисляет факторизацию Холецкого, что необходимо для некоторых методов предобуславливания.

Время построения $O(m^2 n)$

Необходимая память $O(m * n)$

Randomized Nyström Preconditioning

в статье предлагается использовать новый метод решения - **Nyström Preconditioned Conjugate Gradients**. Но для этого сначала нужно построить предобусловленную матрицу P .

Чтобы построить P нужно построить аппроксимированную матрицу для K

Пусть $X \in \mathbb{R}^{n \times l}$ - произвольная тестовая матрица. Обычный Nyström Approximation матрицы A относительно X определяется как:

$$A(X) = (AX)(X^T AX)^\dagger (AX)^T \in S_n^+(\mathbb{R}).$$

Но тут возникает вопрос - как выбирать матрицу X чтобы аппроксимация $A(X)$ была достаточно хорошей. Если выбирать случайно, то хорошей аппроксимации не получится. Тут авторы предлагают **Randomized Nyström Approximation**:

Algorithm 2.1 Randomized Nyström Approximation [21, 37]

Input: Positive-semidefinite matrix $A \in S_n^+(\mathbb{R})$, rank ℓ

Output: Nyström approximation in factored form $\hat{A}_{\text{nys}} = U\hat{\Lambda}U^T$

- 1: $\Omega = \text{randn}(n, \ell)$ ▷ Gaussian test matrix
- 2: $\Omega = \text{qr}(\Omega, 0)$ ▷ Thin QR decomposition
- 3: $Y = A\Omega$ ▷ ℓ matvecs with A
- 4: $\nu = \text{eps}(\text{norm}(Y, \text{'fro'})$ ▷ Compute shift
- 5: $Y_\nu = Y + \nu\Omega$ ▷ Shift for stability
- 6: $C = \text{chol}(\Omega^T Y_\nu)$
- 7: $B = Y_\nu / C$
- 8: $[U, \Sigma, \sim] = \text{svd}(B, 0)$ ▷ Thin SVD
- 9: $\hat{\Lambda} = \max\{0, \Sigma^2 - \nu I\}$ ▷ Remove shift, compute eigs

Берем standard normal test matrix $\Omega \in \mathbb{R}^{n \times l}$, где l вычисляется с помощью *effective dimension*:

$$d_{\text{eff}}(\mu) = \text{tr}(A(A + \mu I)^{-1}) = \sum_{j=1}^n \frac{\lambda_j(A)}{\lambda_j(A) + \mu}.$$

и, соответственно, $l = 2\lceil 1.5d_{\text{eff}}(\mu) \rceil + 1$

И вот теперь из $\hat{A}_{\text{nys}}$ мы получаем U и $\hat{\Lambda}$ и конструируем P

Итак, сингулярное разложение: $\hat{A}_{\text{nys}} = U\hat{\Lambda}U^T$,

за $\hat{\lambda}_l$ обозначаем l -ое собственное значение и тогда **randomized Nyström preconditioner** имеет вид:

$$P = \frac{1}{\hat{\lambda}_\ell + \mu} U(\hat{\Lambda} + \mu I)U^T + (I - UU^T);$$

Robust, randomized preconditioning

Рассматривается два метода, первый для **full-data KRR**, то есть классический KRR, а второй **restricted**, то есть это ограниченный вариант, в котором используется только ограниченное количество базисных функций. Итак первый метод:

RPCholesky preconditioning

снова начинаем с аппроксимации $\hat{A}$ матрицы A с помощью RPCholesky и потом вводим preconditioner P - ничего особого

Algorithm 1 RPCholesky preconditioning
Input: Positive semidefinite matrix $A \in \mathbb{R}^{N \times N}$, right-hand-side vector $y \in \mathbb{R}^N$, regularization coefficient μ , approximation rank r , and tolerance ε .
Output: Approximate solution β , to $(A + \mu I)\beta = y$.
1: $F \leftarrow \text{RPCholesky}(A, r, \min(100, r/10))$. ▷ See Algorithm 4
2: $(U, \Sigma, \sim) \leftarrow \text{ECONOMYSIZESVD}(F)$. ▷ $\hat{A} = U \Sigma^2 U^*$
3: Define primitives for preconditioned conjugate gradient:
 Product: $\beta \mapsto A\beta + \mu\beta$.
 Preconditioner: $\beta \mapsto U[(\Sigma^2 + \mu I)^{-1} - \mu^{-1}I]U^*\beta + \mu^{-1}\beta$.
4: $\beta_* \leftarrow \text{PCG}(\text{Product}, y, \varepsilon, \text{Preconditioner})$. ▷ See Algorithm 3.

где $A(S, :)$, $A(:, S)$, $A(S, S)$ - подматрицы ядровой матрицы A , в первой берем столбцы из S , но все строки, во второй, соответственно, строки из S

Главная идея KRILL - не строить точный $G := (A(S, :) A(:, S))^T$, а заменить на приближение через случайный эмбединг.

Итак сначала строим sparse random sign embedding $\Phi \in \mathbb{R}^{d \times N}$:

$$\Phi = \frac{1}{\sqrt{\zeta}} [\phi_1 \dots \phi_N] \in \mathbb{R}^{d \times N}.$$

где $d = 2k$, $\zeta = \lceil \log(k+1) \rceil$, а столбцы $\phi_1 \dots \phi_N$ имеют ровно ζ ненулевых значений и эти ζ позиций выбираются равномерно случайно среди $\{1, \dots, d\}$, а значения в этих позициях - ± 1

Следующий шаг - сжимаем подматрицу $A(:, S)$:

$$B = \Phi A(:, S) \in \mathbb{R}^{d \times k}.$$

KRILL preconditioning

тут мы сначала уменьшаем задачу

в обычном KRR (тут обозначили за β то что раньше было α : $(A + \mu I)\alpha = y$):

$$f(x) = \sum_{i=1}^N \beta_i K(x^{(i)}, x).$$

в restricted случае мы выбираем индексы $S = \{s_1, \dots, s_k\}$ и строим ограниченный предиктор:

$$\hat{f}(x; \hat{\beta}) = \sum_{j=1}^k \hat{\beta}_j K(x^{(s_j)}, x).$$

где $x^{(s_1)}, \dots, x^{(s_k)}$ называются центрами, тогда исходная система переписывается в следующем виде:

$$(A(S, :) A(:, S) + \mu A(S, S)) \hat{\beta} = A(S, :) y.$$

Далее используем приближение $\hat{G} = B^T B$ и вот итоговый предобуславливатель готов:

$$P = B^T B + \mu A(S, S) = (\Phi A(:, S))^T (\Phi A(:, S)) + \mu A(S, S)$$

Algorithm 2 KRILL preconditioning

Input: Positive semidefinite matrix $A \in \mathbb{R}^{N \times N}$, right-hand-side vector $y \in \mathbb{R}^N$, regularization coefficient μ , centers S , and tolerance ε .
Output: Approximate solution $\hat{\beta}$, to $(A(S, :) A(:, S) + \mu A(S, S)) \hat{\beta} = A(S, :) y$.
1: $H \leftarrow \mu A(S, S)$
2: $H \leftarrow H + N \varepsilon_{\text{mach}} \text{tr}(A(S, S)) I$. ▷ $\varepsilon_{\text{mach}} \approx 2 \times 10^{-16}$ in double precision.
3: $k \leftarrow |S|$.
4: $\Phi \leftarrow \text{SPARSESIGNEMBEDDING}(d = 2k, N, \zeta = \lceil \log(k+1) \rceil)$. ▷ See Algorithm 5.
5: $B \leftarrow \Phi A(:, S)$.
6: $P \leftarrow B^T B + H$.
7: $C \leftarrow \text{Cholesky}(P)$.
8: Define primitives for preconditioned conjugate gradient:
 Product: $\hat{\beta} \mapsto A(:, S)(A(S, :) \hat{\beta}) + H \hat{\beta}$.
 Preconditioner: $\hat{\beta} \mapsto C^{-1}(C^{-*} \hat{\beta})$.
9: $\hat{\beta}_* \leftarrow \text{PCG}(\text{Product}, A(S, :) y, \varepsilon, \text{Preconditioner})$. ▷ See Algorithm 3.

Adaptive Factorized Nyström Preconditioner

тут мы также выбираем $X_k = \{x^{(s_1)}, \dots, x^{(s_k)}\}$ - landmark points, но только не чтобы ограничить задачу как и в KRILL, а исключительно для построения предобуславливателя. с помощью алгоритма 3.4 оцениваем сколько этих landmark points нужно чтобы аппроксимация была достаточно хороша

Algorithm 3.4 Nyström rank estimation

```

1: Input: Dataset  $X$  with size  $n$ , subsample size  $m$ , and kernel function  $\mathcal{K}(x, y)$ 
2: Output: Approximate Nyström rank  $k$ 
3: Randomly subsample a subset  $X_m$  of  $m$  points from  $X$  and scale the coordinates of  $X_m$  by  $(m/n)^{1/d}$ 
4: Form the  $m \times m$  matrix  $\mathbf{K}_{X_m, X_m}$ 
5: Find the Nyström rank  $r$  such that the relative Nyström approximation error for  $\mathbf{K}_{X_m, X_m}$  with FPS sampling implemented in Algorithm 4.1 falls below 0.1
6: if  $k \geq 2000$  then
7:   Return  $k = rn/m$ 
8: else
9:   Compute eigenvalues of  $\mathbf{K}_{X_m, X_m}$  associated with the unscaled data points
10:  Return  $k$ , the number of eigenvalues greater than 0.1 $\mu$ 
11: end if
    
```

Далее считаем что X_k идут первыми $\rightarrow$ переписываем K в блочном виде:

$$K = \begin{bmatrix} K_{11} & K_{12} \\ K_{12}^T & K_{22} \end{bmatrix}$$

где $K_{11} = K(X_k, X_k)$, $K_{12} = K(X_k, X_r)$, $K_{22} = K(X_r, X_r)$

Далее к K_{11} применяем Cholesky:

$$L = \text{chol}(K_{11} + \mu I)$$

Следующий шаг:

$$S = K_{22} + \mu I - K_{12}^T (K_{11} + \mu I)^{-1} K_{12}$$

Строим матрицу $G^T G \approx S^{-1}$ с помощью алгоритма 3.1:

Algorithm 3.1 Factorized Sparse Approximate Inverse (FSAI)

```

1: Input: A symmetric positive definite matrix  $\mathbf{K}$ , the number of nonzeros on each row  $w$ , a lower triangular sparsity pattern  $\mathbf{S} \in \mathbb{R}^{n \times n}$  which is a zero-one matrix with at most  $w$  ones on each row indicating the positions of  $w$ -nearest neighbors of each data point.
2: for  $i = 1$  to  $n$  do
3:   Extract the non-zero pattern  $s_i$  from the  $i$ th row of  $\mathbf{S}$  with length  $w \ll n$ 
4:   Compute  $\mathbf{G}_{i, s_i} = \frac{\mathbf{e}_w^T (\mathbf{K}_{s_i, s_i})^{-1}}{\sqrt{\mathbf{e}_w^T (\mathbf{K}_{s_i, s_i})^{-1} \mathbf{e}_w}}$ 
5: end for
6: Return:  $\mathbf{G}$ 
    
```

В алгоритме 3.2 написано как именно мы выбираем эти k штук landmark points.

Algorithm 3.2 Adaptive Factorized Nyström (AFN) preconditioner construction

```

1: Input: Dataset  $X = \{\mathbf{x}_i\}_{i=1}^n$  for  $\mathbf{x}_i \in \mathbb{R}^d$ , Kernel function  $\mathcal{K}(\cdot, \cdot)$ , regularization parameter  $\mu$ , estimated rank  $k$  returned by Algorithm 3.4.
2: if  $d \leq 10$  then
3:   Select  $k$  landmark points denoted by  $X_k$  according to FPS Algorithm 4.1.
4: else
5:   Select  $k$  landmark points denoted by  $X_k$  using uniform sampling.
6: end if
7: Perform Cholesky factorization:  $\mathbf{L} = \text{Chol}(\mathbf{K}_{11} + \mu \mathbf{I})$  where  $\mathbf{K}_{11} = \mathcal{K}(X_k, X_k)$ 
8: Invoke Algorithm 3.1 to compute  $\mathbf{G} = \text{FSAI}(\mathbf{K}_{22} + \mu \mathbf{I} - \mathbf{K}_{12}^T (\mathbf{K}_{11} + \mu \mathbf{I})^{-1} \mathbf{K}_{12})$  where  $\mathbf{K}_{12} = \mathcal{K}(X_k, X_r)$ ,  $\mathbf{K}_{22} = \mathcal{K}(X_r, X_r)$ ,  $X_r = X \setminus X_k$ .
9: Return: Matrices  $\mathbf{L}$  and  $\mathbf{G}$ 
    
```

И наконец итоговый предобуславливатель:

$$M = \begin{bmatrix} L & 0 \\ K_{12}^T L^{-T} & G^{-1} \end{bmatrix} \begin{bmatrix} L^T & L^{-1} K_{12} \\ 0 & G^{-T} \end{bmatrix}$$

Algorithm 3.2 Adaptive Factorized Nyström (AFN) preconditioner construction

```

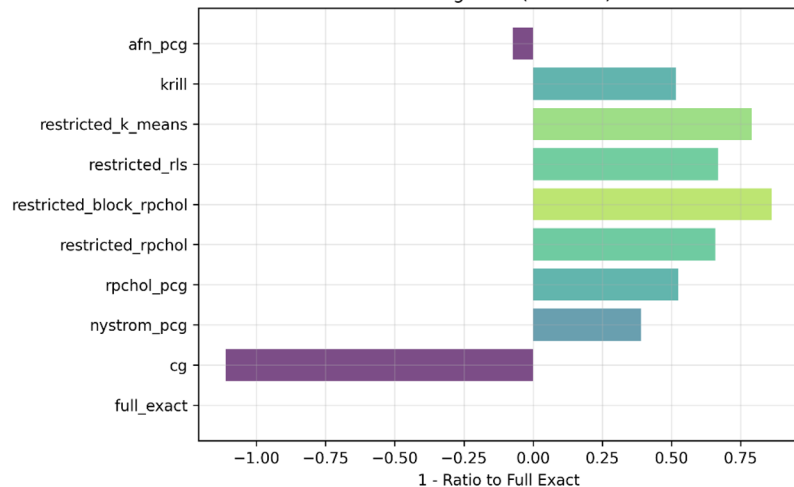
1: Input: Dataset  $X = \{\mathbf{x}_i\}_{i=1}^n$  for  $\mathbf{x}_i \in \mathbb{R}^d$ , Kernel function  $\mathcal{K}(\cdot, \cdot)$ , regularization parameter  $\mu$ , estimated rank  $k$  returned by Algorithm 3.4.
2: if  $d \leq 10$  then
3:   Select  $k$  landmark points denoted by  $X_k$  according to FPS Algorithm 4.1.
4: else
5:   Select  $k$  landmark points denoted by  $X_k$  using uniform sampling.
6: end if
7: Perform Cholesky factorization:  $\mathbf{L} = \text{Chol}(\mathbf{K}_{11} + \mu \mathbf{I})$  where  $\mathbf{K}_{11} = \mathcal{K}(X_k, X_k)$ 
8: Invoke Algorithm 3.1 to compute  $\mathbf{G} = \text{FSAI}(\mathbf{K}_{22} + \mu \mathbf{I} - \mathbf{K}_{12}^T (\mathbf{K}_{11} + \mu \mathbf{I})^{-1} \mathbf{K}_{12})$  where  $\mathbf{K}_{12} = \mathcal{K}(X_k, X_r)$ ,  $\mathbf{K}_{22} = \mathcal{K}(X_r, X_r)$ ,  $X_r = X \setminus X_k$ .
9: Return: Matrices  $\mathbf{L}$  and  $\mathbf{G}$ 
    
```

Эксперименты

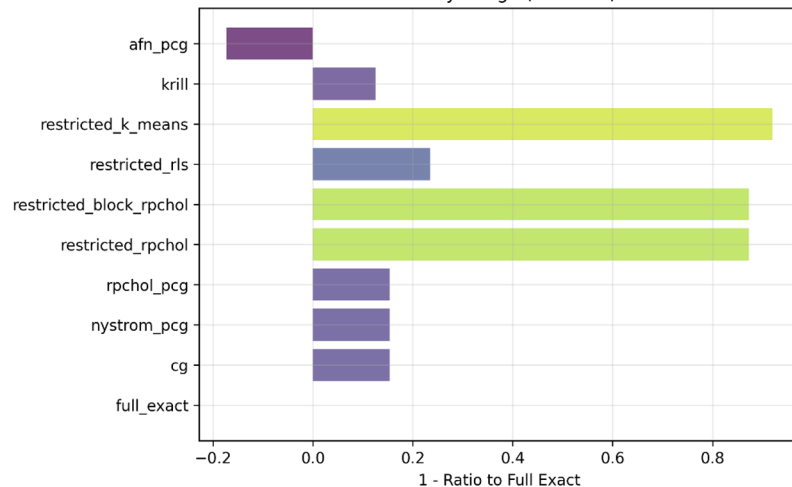
Составили параметрическую сетку из типов рядов и степени регулязации. Для одного и того же эффективного ранга посчитали trade-off между временем обучения - предсказания - памяти и потерянным качеством.

PROTEIN Dataset

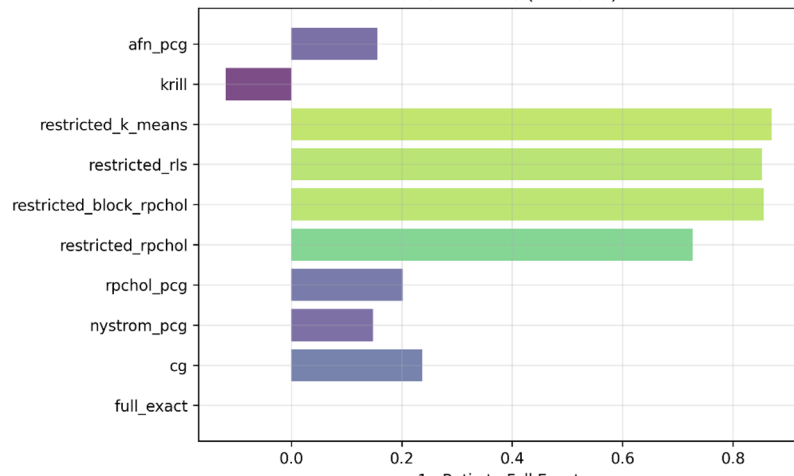
Learning Time (1 - Ratio)



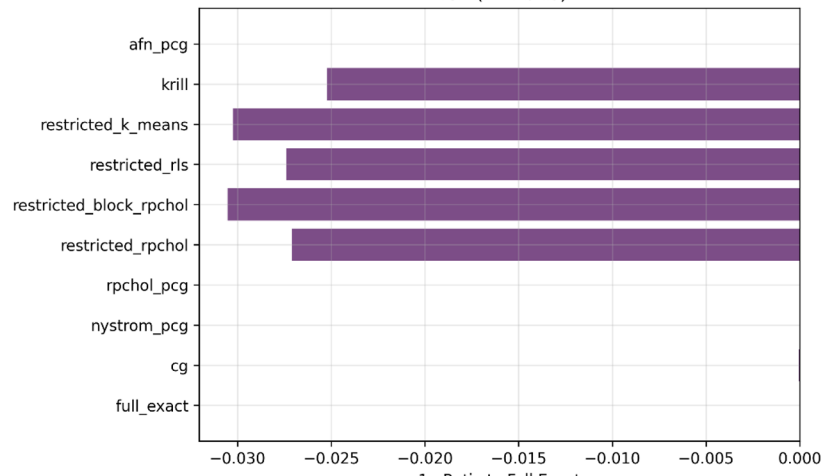
Memory Usage (1 - Ratio)



Prediction Time (1 - Ratio)



MSE (1 - Ratio)



Выводы

Все методы почти не теряют качество (не более 3%)

Restricted KRR очень хорошо экономит время и память в независимости от метода выбора опорных векторов, но `grchol` и `k-means` чуть лучше.

Методы на основании PCG вообще не имеют хоть какие-нибудь существенные потери в качестве, но очень сложны в реализации и при определенных параметрах могут вычисляться столько же сколько и полная задача, а иногда и больше.

Nystrom PCG и RP Cholesky PCG наилучшие методы, если вы хотите наилучшее качество но и значительно сэкономить ресурсы.

Restricted-KRR - большая экономия + хорошее качество
PCG - хорошая экономия + отличное качество

Распределение

Матросова - Restricted KRR

Зориков - PCG

Ссылки

Наш github:

<https://github.com/lisssica/Scalable-KRR>

Статьи:

<https://arxiv.org/abs/2207.06503>

<https://www.arxiv.org/abs/2506.17556>

https://proceedings.neurips.cc/paper_files/paper/2024/file/257c140d25d1f7fc10f8ae5e17296299-Paper-Conference.pdf

<https://arxiv.org/pdf/2110.02820>

<https://arxiv.org/abs/2304.05460>

<https://arxiv.org/abs/2304.12465>